King Fahd University of Petroleum & Minerals

College of Computing and Mathematics

SEC 524 – Computer and Network Forensics

OPERATIONAL TECHNOLOGY FORENSICS

Reconstructing an Unauthorised PLC Setpoint Change from Modbus/TCP Traffic and Controller Logs in an Isolated Virtual Testbed

Coursework: Digital Forensics in Emerging Technologies

Prepared by:

Student 1 Full Name – ID: Student 1 ID

Student 2 Full Name – ID: Student 2 ID

Instructor: Instructor Name

Course: 251-SEC-524-01

November 2026

Contents

Executive Summary 3
1. Introduction and Investigation Scope 3
2. Technology and Threat Context 4
2.1 Architecture, actors and data flows 4
2.2 Incident scenario and threats 4
2.3 How this differs from conventional forensics 5
3. Evidence Sources and Forensic Challenges 5
3.1 Evidence sources and volatility 5
3.2 Main challenges and why they matter 6
4. Proposed Forensic Framework 7
4.1 Prepare (stages 1–3) 7
4.2 Preserve (stages 4–6) 8
4.3 Analyse (stages 7–9) 8
4.4 Interpret and improve (stages 10–13) 8
5. Practical Investigation 8
5.1 Scenario and investigative question 8
5.2 Environment 8
5.3 Acquisition procedure 9
5.4 Tools and why we chose them 9
6. Findings and Evidence Analysis 10
6.1 Negative control (E1) 10
6.2 The write and its corroboration (E2, E3) 10
6.3 Hypothesis testing 11
6.4 Validation 11
7. Discussion and Limitations 12
8. Legal, Ethical and Privacy Considerations 13
9. Forensic-Readiness Recommendations 13
10. Conclusion 14
References 14
Appendix A – Commands Used 16
Appendix B – Hash Manifest and Chain of Custody 16
Appendix C – Raw Decode Output 17
Appendix D – Selected Screenshots 18
Appendix E – Supporting Files and AI-Use Statement 19

Executive Summary

This report looks at one forensic problem in Operational Technology (OT): somebody changed a setpoint inside a programmable logic controller (PLC) over Modbus/TCP, and there was no approved change request for it. We wanted to know what an investigator can actually prove in that situation, and what stays unknown. To answer this, we built an isolated VirtualBox testbed with a software PLC and a mock engineering workstation, recorded a read-only baseline, and then performed one controlled write that changed holding register 16 from 50 to 90.

The evidence (two packet captures, the PLC audit log, a console log and a SHA-256 manifest) was preserved, re-verified and analysed with two independent methods: tshark and our own Python decoder working directly on the raw PCAP bytes. All three sources agree. The write request left 10.20.30.20 at 16:31:30.220 UTC, the PLC echoed it 6.9 ms later, the audit log recorded the change $50 \rightarrow 90$ 3.5 ms after that, and a read 3.7 s later returned 90. The baseline capture contains no write at all. However, the evidence identifies a host address and not a person, because Modbus has no user authentication, and both timing sources come from the same clock. We also propose a 13-stage OT-adapted framework and give readiness recommendations to close these gaps.

1. Introduction and Investigation Scope

Most forensic training assumes that the investigator can take a computer offline and image its disk. In OT this is often not possible. NIST describes OT as programmable systems that interact with the physical world, where availability and safety come first [1]. Rebooting a controller or running an active scan on a live control network could stop a process or harm people, so the investigator must collect evidence in a way that does not disturb the plant [1], [8].

Our scope is deliberately narrow, as the coursework brief recommends. We focus on a single Modbus/TCP holding-register write to one controller. The investigative question is:

Can independent artefacts show which command was sent, from where, when, and whether the PLC accepted it, and where do the limits of attribution lie?

Out of scope are malware analysis, attribution to an external threat actor, and any work on real industrial equipment. All testing was done on simulated equipment in an isolated virtual network that we built and own.

2. Technology and Threat Context

2.1 Architecture, actors and data flows

A typical small OT cell has an engineering workstation used to configure and maintain controllers, a PLC that runs the control logic, an HMI where operators watch and adjust values, and often a historian that stores process trends [1]. Modbus is one of the oldest and most common protocols between these parts. A Modbus/TCP message is a 7-byte MBAP header (transaction ID, protocol ID, length and unit ID) followed by a PDU that holds the function code and data [5], [6]. Function code 03 reads holding registers and function code 06 writes a single register. For a successful FC06 write the server simply echoes the request back [5]. Modbus has no authentication or integrity protection, so any host that can reach the PLC can change a register [10].

Figure 1 shows the testbed used in this work. It reproduces the minimum data flow that matters for our question: an engineering host sends reads and writes to a controller, and the controller keeps an audit trail.

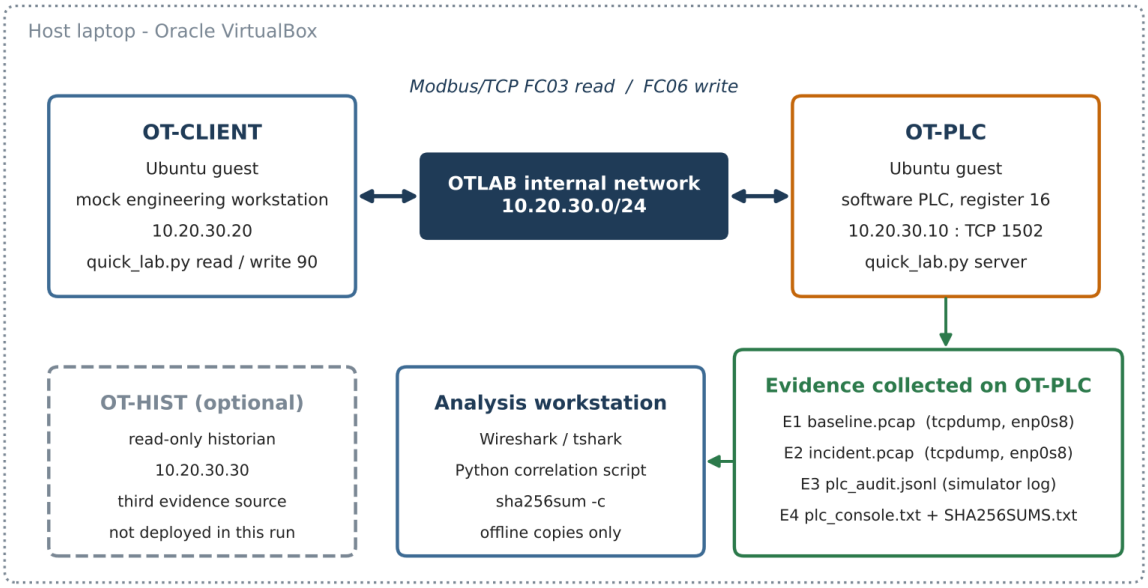


Figure 1. Virtual OT testbed and evidence collection points (E1–E4).

2.2 Incident scenario and threats

In our case notes, register 16 is a process setpoint with normal value 50. No change request was approved for the test period. A write that moves it to 90 is therefore treated as unauthorised. MITRE ATT&CK for ICS lists this behaviour as Modify Parameter (T0836) [7]. The main attack surfaces are the unauthenticated Modbus service, the engineering workstation (which can be

misused or compromised), remote maintenance paths, and weak network segmentation [1], [10]. Table 1 maps the relevant events to the artefacts they should leave behind.

Event / attack	Expected artefacts	Where they live
Reconnaissance read (FC03)	Request/response pair, register value	Network capture; PLC audit log
Parameter write (FC06, T0836)	Write PDU with register and value; echoed response; before/after state	Network capture; PLC log; historian trend
Spoofed or replayed write	Same PDU bytes from unexpected MAC/IP or TCP sequence	Switch/ARP tables; capture at TAP
Misuse of engineering host	Logon, shell history, engineering software activity	Workstation logs and memory
Covering tracks	Gaps in logs, rotated files, clock changes	Log forwarder; NTP logs; file metadata

Table 1. Mapping of incident events to expected forensic artefacts.

2.3 How this differs from conventional forensics

Three things make OT different. First, the "victim" device is usually a PLC with little storage, no standard file system and proprietary firmware, so disk imaging is not an option [8], [9]. Second, the meaning of the evidence depends on engineering context: the number 90 means nothing until a register map tells us it is, for example, a temperature limit. Third, the safety of the process limits what the investigator may do. In IT forensics we try to collect as much as possible. In OT we collect as much as is safe [1], [9].

3. Evidence Sources and Forensic Challenges

3.1 Evidence sources and volatility

Table 2 lists the evidence sources for this kind of incident, ordered roughly from most to least volatile. The order matters because an investigator who exports historian data first may lose the controller state or the packets in a rolling capture buffer [2], [3].

Source	Volatility	Acquisition approach	What it can prove
PLC run-time state (register values)	Very high – lost on restart or next write	Read-only query or vendor diagnostics, under OT supervision	Current state only
Network traffic (SPAN/TAP or endpoint)	High – exists only if captured live	Passive capture (tcpdump), hash at stop	Command, function, register, value, IP/port, time
PLC / simulator audit log	Medium – limited size, may rotate	Export copy, hash, keep original	Whether the controller applied the change
Engineering workstation	Medium – RAM high, logs medium	Triage, log export, memory image if approved	Account, process, session behind an IP
Historian / HMI trends	Low–medium – sampled, buffered	Read-only export	Process effect over time

Source	Volatility	Acquisition approach	What it can prove
Firewall, switch, DHCP logs	Low–medium – rolling	Configuration-preserving export	IP-to-device mapping, path

Table 2. Evidence sources for an unauthorised Modbus write, in order of volatility.

3.2 Main challenges and why they matter

Live versus static acquisition. A running PLC cannot be switched off and imaged without stopping the process. So most evidence must be collected live, and live collection always changes something [8]. The investigator has to record what was touched and justify it, otherwise a defence lawyer can question whether the evidence was altered.

Attribution and identity. Modbus carries a unit ID, not a user name. The best a packet can say is "this IP and port sent this command". NAT, shared engineering accounts and spoofed addresses all weaken the link from IP to person [10]. This is the biggest gap between what an investigator can observe and what a court usually wants to hear.

Time synchronisation. PLCs often have drifting clocks or none at all, historians sample at fixed intervals, and capture hosts may be in different time zones. If times are not normalised to UTC and offsets are not measured, the investigator can put events in the wrong order and reach the wrong conclusion [2], [9].

Proprietary formats and tool validation. Many controllers use vendor-specific logs and protocols. Even for open Modbus, tools make assumptions. We found in our own analysis that Wireshark could not label packets as query or response on a non-standard port (Section 6.4). An investigator who does not validate the tool may misreport direction.

Incomplete visibility. A SPAN port can drop packets and a capture only sees one path. Absence of a packet in the capture is therefore not proof that nothing happened [1].

Anti-forensics. An attacker can delete workstation logs, write a value back to its original number, change device clocks or blend in using a legitimate engineering account. Countermeasures are remote log forwarding, independent passive monitoring, NTP with logged offsets, and comparison with a configuration baseline [1], [9].

Integrity and chain of custody. A hash proves that a copy has not changed since it was hashed. It does not prove that the capture was complete or that the source was genuine. This is why custody records, acquisition notes and corroboration from a second source are still needed [3].

4. Proposed Forensic Framework

We started from the NIST SP 800-86 forensic process (collection, examination, analysis, reporting) [2] and the OSCAR network-forensics model taught in the course (Obtain information, Strategise, Collect evidence, Analyse, Report) [15]. Both assume that collecting evidence is low risk for the system under investigation. In OT that assumption is false, so we adapted them into 13 stages in four phases (Figure 2). The orange stages are the ones we changed or added.

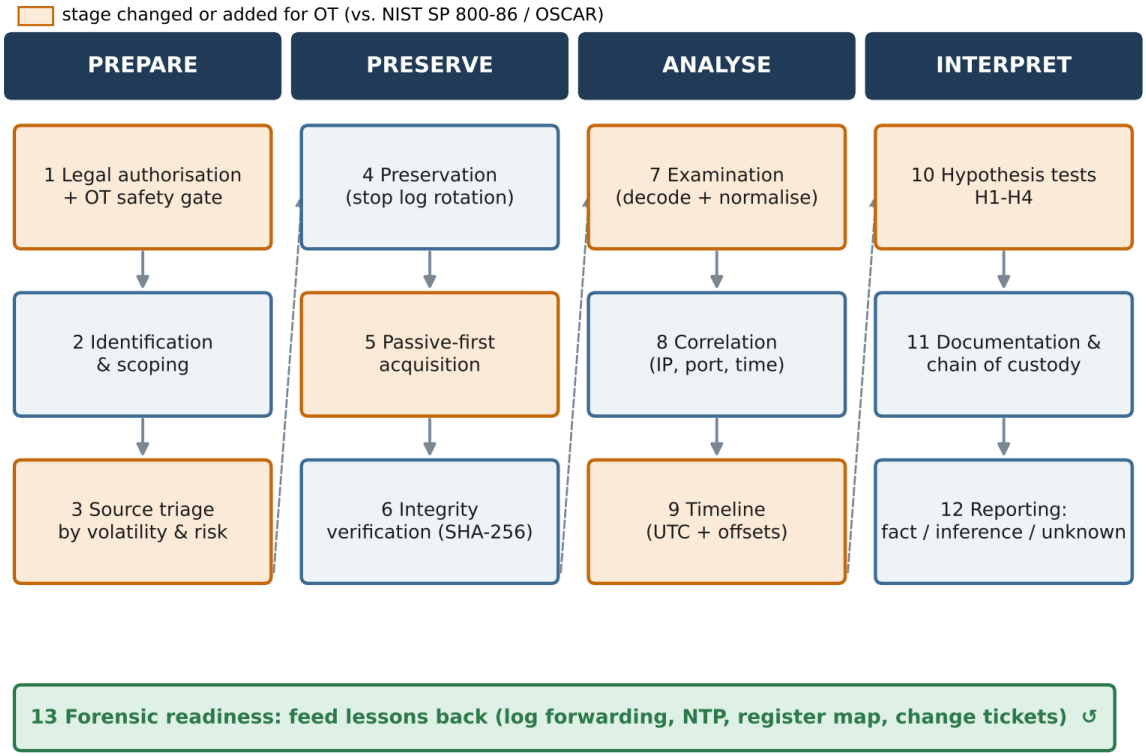


Figure 2. OT-adapted forensic investigation framework (13 stages).

4.1 Prepare (stages 1–3)

Stage 1 gets legal authorisation and also an **OT safety gate**: the plant owner and a process engineer must agree on what the team may touch, and on the conditions under which collection stops. **Stage 2** preserves the original alert and records alternative explanations from the start, such as a technician, an HMI script or a scheduled task. **Stage 3** ranks evidence sources by volatility, by evidential value and by the **risk to the process** of collecting them. This third factor is our main change, because a source that is valuable but risky (for example, a PLC memory dump) may have to wait or be skipped.

4.2 Preserve (stages 4–6)

Stage 4 stops rotation of existing captures and logs. **Stage 5** follows a **passive-first** rule: use existing captures, mirror ports and read-only exports before anything that sends traffic to the controller. Active reads are allowed only when stage 1 has approved them. **Stage 6** hashes every file with SHA-256 at the point of acquisition, records size, tool and operator, and re-verifies the hash after every transfer [3].

4.3 Analyse (stages 7–9)

Stage 7 decodes the protocol and normalises all logs into one schema (UTC time, source, event, register, value). For OT we add **protocol semantics**: decoding the register number and value is not enough, so the investigator must link them to the register map and engineering units. **Stage 8** correlates sources using IP, TCP port and time, and keeps device identity separate from account identity. **Stage 9** builds the timeline in UTC and records the **clock source and offset** of each evidence item, so that the confidence of ordering is explicit.

4.4 Interpret and improve (stages 10–13)

Stage 10 tests competing hypotheses (H1–H4 in Section 6.3) instead of confirming the first story. **Stage 11** keeps custody forms, notes and the link between each claim and its exhibit. **Stage 12** reports findings in three classes: observed fact, reasonable inference and unknown. **Stage 13** feeds lessons back into forensic readiness (Section 9), which NIST SP 800-61r3 treats as part of continuous risk management [4].

5. Practical Investigation

5.1 Scenario and investigative question

The practical part applies stages 3 to 12 of the framework to a controlled incident. The scenario is the one in Section 2.2, and the question is whether the network and controller evidence can reconstruct the write to register 16 and support or reject hypotheses H1–H4.

5.2 Environment

Two Ubuntu virtual machines were created in Oracle VirtualBox on one host laptop. Each VM has a second adapter attached to a VirtualBox Internal Network named `OTLAB` (Appendix D, Figure D1). The VM **OT-PLC** (hostname `APPServer`, 10.20.30.10/24) runs `quick_lab.py server`. This is an 88-line Python Modbus/TCP server that holds register 16 (initial value 50), answers FC03 and FC06, listens on TCP 1502 and writes one JSON line per request to

`plc_audit.jsonl`. The VM **OT-CLIENT** (hostname `Ubuntu2`, 10.20.30.20/24) runs the same script in client mode. Connectivity was checked with ping in both directions (0% loss). We used port 1502 instead of 502 so the server could run without root privileges.

5.3 Acquisition procedure

All capture was done on OT-PLC with `tcpdump -i enp0s8 -s 0` and the filter `tcp port 1502`, so full packets were saved and only the lab protocol was recorded. The run on 26 October 2026 had two phases:

- **Baseline (negative control):** capture to `baseline.pcap`, then three reads from OT-CLIENT one second apart.
- **Incident:** new capture to `incident.pcap`, then read, `write 90`, read. The client printed "Register 16 = 50", "Write acknowledged: 90" and "Register 16 = 90".

After each capture, tcpdump was stopped. The simulator was then stopped with Ctrl+C so the audit log was closed before hashing. SHA-256 digests of the four files were written to `SHA256SUMS.txt` on OT-PLC and checked with `sha256sum -c` (all OK). The folder was archived as `CourseWork.zip` and shared with the second team member, who re-checked the manifest on the analysis workstation on 11 November 2026. All four files matched (Appendix B). The full commands are in Appendix A.

ID	File	Size (bytes)	SHA-256 (first 16 hex)	Content
E1	baseline.pcap	2,601	557fcdac0c507317…	30 frames, 16:27:40.846–16:27:42.997 UTC
E2	incident.pcap	2,602	e919a17b09329b85…	30 frames, 16:31:17.589–16:31:33.947 UTC
E3	plc_audit.jsonl	1,012	fa48dcc0bcd6c439…	6 records (5 READ, 1 WRITE_APPLIED)
E4	plc_console.txt	1,074	c85ff2b1b6bdfe3c…	Start banner + same 6 records

Table 3. Evidence inventory (full hashes and custody in Appendix B).

5.4 Tools and why we chose them

tcpdump is passive and writes standard PCAP files, which fits stage 5. **Wireshark/tshark** (re-analysis with TShark 4.2.2) has a mature Modbus/TCP dissector and display filters such as `modbus.func_code == 6` [11]. To avoid depending on one tool, we wrote a short **Python decoder** (`correlate_evidence.py`, standard library only). It reads the PCAP bytes directly, decodes every MBAP header and PDU, re-checks the hashes, and matches each audit record with its request and response packets. **sha256sum** was used for integrity, as recommended by ISO/IEC 27037 practice [3].

6. Findings and Evidence Analysis

6.1 Negative control (E1)

Filtering E1 with `modbus.func_code == 6` returned zero packets in tshark, and our decoder also counted zero FC06 frames. E1 holds three TCP sessions, each with one FC03 request for register 16 and one response with value 50 (0x0032). This gives us a clean picture of "normal" traffic to compare with the incident.

6.2 The write and its corroboration (E2, E3)

In E2, frame 14 is a TCP segment from 10.20.30.20:45712 to 10.20.30.10:1502 with the payload `01 00 00 00 06 01 06 00 10 00 5A`. The MBAP header gives transaction ID 1, protocol 0, length 6 and unit 1. The PDU is function 06, register address 0x0010 (16), value 0x005A (90). Frame 16, 6.87 ms later, carries the identical 12 bytes in the opposite direction. According to the Modbus specification this echo means the write was accepted [5]. Frame 26 then answers a new read with value 0x005A, so the register still held 90 3.7 s later.

E3 contains exactly one `WRITE_APPLIED` record: client 10.20.30.20, function 6, register 16, argument 90, before 50, after 90, at 16:31:30.230534 UTC. Table 4 merges the two sources. Every one of the six audit records lines up with exactly one request/response pair, and the IP addresses match each time.

UTC time (26 Oct 2026)	Source	Event	Detail
16:31:17.589642	E2 #4	FC03 request	10.20.30.20:45510 → PLC, reg 16
16:31:17.593918	E2 #6	FC03 response	value 50
16:31:17.594328	E3	READ	before 50 / after 50
16:31:30.220150	E2 #14	FC06 request	10.20.30.20:45712 → PLC, reg 16 = 90
16:31:30.227024	E2 #16	FC06 echo	identical bytes, write accepted
16:31:30.230534	E3	WRITE_APPLIED	before 50 / after 90
16:31:33.942837	E2 #24	FC03 request	10.20.30.20:45714 → PLC, reg 16
16:31:33.944087	E2 #26	FC03 response	value 90
16:31:33.944305	E3	READ	before 90 / after 90

Table 4. Reconstructed incident timeline (packet times from E2, log times from E3).

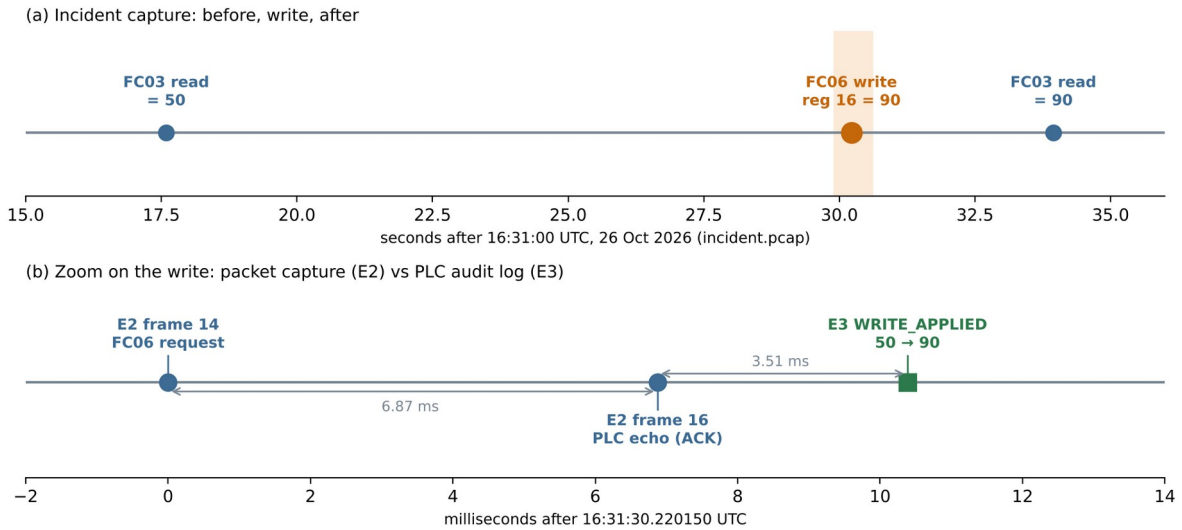


Figure 3. Incident timeline; (b) shows the order request → echo → log entry.

The ordering is always the same: request, then response, then log entry. The log line is written 0.2 to 0.8 ms after the response for reads and 3.5 ms after for the write. This fits the simulator code, which sends the reply before it appends to the log. We also noticed two details that matter for analysis. First, the transaction ID is 1 in every message, so it cannot be used to pair requests and responses across sessions. We paired them by TCP connection instead, since each command used a new source port. Second, the write response is byte-for-byte equal to the request, so only the direction (source port 1502) tells them apart.

6.3 Hypothesis testing

Hypothesis	Evidence	Result
H1: an FC06 write to register 16 crossed the network	E2 #14 decoded by tshark and Python; absent from E1	Supported (observed)
H2: the PLC accepted the write and its state changed	E2 #16 echo; E3 WRITE_APPLIED 50→90; E2 #26 reads 90	Supported (observed)
H3: a specific person made the change	Only IP 10.20.30.20 and an ephemeral port; Modbus carries no user identity; no workstation logs were acquired	Not supported – host only
H4: the change was authorised maintenance	Scenario case notes record no approved change request	Rejected by scenario records (inference)

Table 5. Competing hypotheses and outcome.

6.4 Validation

We validated the findings in three ways. **(1) Two decoders.** tshark and our Python script produced the same frame numbers, IPs, ports, function codes, registers and values for all twelve Modbus PDUs (Appendix C). **(2) Two evidence types.** The network capture and the controller log are produced by different mechanisms (the kernel capture path and the application), and they agree on

every event, order and value. **(3) Ground truth.** Because we ran the incident ourselves, we know the real sequence was read, write 90, read. The reconstruction matches it exactly, with no missing or extra events.

Validation also exposed a tool problem. With the usual "Decode As" setting, tshark labelled frames 14 and 16 as "unknown … Unable to classify as query or response", because the dissector decides direction by comparing against port 502. Setting the preference `-o mbtcp.tcp.port:1502` fixed this, and the frames were correctly labelled Query and Response. Although this is a small issue, it shows why stage 7 of the framework requires tool settings to be validated and recorded.

7. Discussion and Limitations

We separate the results into three classes, as stage 12 requires. **Observed facts:** an FC06 write of 90 to register 16 was sent from 10.20.30.20, it was echoed, the simulator logged 50→90, and a later read returned 90. **Reasonable inferences:** the write came from the OT-CLIENT VM, because that VM held 10.20.30.20 in the isolated network, and it was unauthorised, because the case notes record no change request. **Unknowns:** which person typed the command, and whether any physical process would have been affected.

Several limitations affect how far these results can be generalised:

- E2 and E3 were both produced on OT-PLC and use the same system clock. Their agreement shows the records are consistent, but it does not independently confirm the absolute time. An external NTP-synchronised source or a historian on a separate VM would be needed for that.
- The capture was taken on the PLC host itself, not from a separate SPAN or TAP, so it shows what the PLC received rather than everything on the wire.
- The software PLC has one register, no control logic and no vendor firmware. Real controllers have proprietary logs and may reject or remap writes, so the method transfers but the artefact formats do not.
- No workstation artefacts (shell history, authentication logs, memory) were acquired from OT-CLIENT, so H3 cannot be tested beyond the IP level.
- The dataset is small (60 frames). It is enough to demonstrate the method, but it does not test how the framework scales to busy networks with constant polling.

8. Legal, Ethical and Privacy Considerations

In Saudi Arabia, unauthorised access to an information system or interference with its data falls under the Anti-Cyber Crime Law (Royal Decree M/17, 1428H) [13]. Packet evidence alone does not show criminal intent, however, so an investigator should describe the act and leave the legal classification to the competent authorities. For operators within its scope, such as critical national infrastructure, the NCA Operational Technology Cybersecurity Controls (OTCC-1:2022) require OT event logging, monitoring and incident management. These controls support forensic readiness, but they do not apply automatically to every organisation [12].

Workstation logs, remote-access records and shift rosters usually contain personal data about employees. The Personal Data Protection Law (Royal Decree M/19, amended by M/148) requires a lawful purpose, data minimisation and protection of that data, and it restricts transfers outside the Kingdom. This is relevant when a vendor or a cloud backup holds the logs abroad [14]. The investigator should collect only what the incident needs, restrict access and encrypt case files.

Chain of custody must record who collected each item, when, how, every transfer and where it is stored, with hashes at each step [3]. Our custody table is in Appendix B. On the ethical side, safety comes before evidence: no active scanning or writes should be sent to a live controller only to complete an investigation. In this project all writes were sent to our own simulator inside an internal network with no physical equipment attached.

9. Forensic-Readiness Recommendations

- Deploy a passive TAP or SPAN at the engineering-to-controller conduit with a rolling, hashed PCAP buffer, so that captures already exist when an alert comes in.
- Forward PLC, HMI and workstation logs to a tamper-resistant collector in real time, so that local deletion does not erase the record.
- Synchronise all capable devices to one NTP source, and log the measured offset for devices that cannot synchronise.
- Maintain an up-to-date register map with engineering units, and link every planned write to a change ticket, so that "unusual" can be separated from "authorised".
- Require named accounts and multi-factor login on engineering workstations and collect their shell and authentication logs. This is the only practical way to close the IP-to-person gap in H3.

- Configure Modbus tooling for non-standard ports and validate dissectors on a test capture before real incidents (Section 6.4).
- Practise the procedure regularly in a lab like ours with IT, OT engineering and legal staff together.

10. Conclusion

This project showed that an unauthorised PLC setpoint change can be reconstructed with confidence when network captures and controller logs are preserved and correlated. In our testbed, two independent decoders and two types of evidence agreed on every event, and the reconstruction matched the known ground truth. The same work also showed the limits. The evidence identifies a host, not a person, and agreement between two records that share one clock is consistency, not independent proof of time. Our OT-adapted framework keeps the core of NIST and OSCAR, but adds a safety gate, passive-first collection, protocol semantics and explicit clock handling. These additions are what make the investigation both safe for the plant and defensible in its conclusions.

References

[1] K. Stouffer et al., "Guide to Operational Technology (OT) Security," NIST SP 800-82 Rev. 3, Sep. 2023, doi: 10.6028/NIST.SP.800-82r3.

[2] K. Kent, S. Chevalier, T. Grance, and H. Dang, "Guide to Integrating Forensic Techniques into Incident Response," NIST SP 800-86, Aug. 2006, doi: 10.6028/NIST.SP.800-86.

[3] Information technology — Security techniques — Guidelines for identification, collection, acquisition and preservation of digital evidence, ISO/IEC 27037:2012, 2012.

[4] A. Nelson, S. Rekhi, M. Souppaya, and K. Scarfone, "Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile," NIST SP 800-61 Rev. 3, Apr. 2025, doi: 10.6028/NIST.SP.800-61r3.

[5] Modbus Organization, "MODBUS Application Protocol Specification V1.1b3," 2012. [Online]. Available: https://www.modbus.org/specs.php

[6] Modbus Organization, "MODBUS Messaging on TCP/IP Implementation Guide V1.0b," 2006. [Online]. Available: https://www.modbus.org/specs.php

[7] MITRE, "Modify Parameter (T0836)," ATT&CK for ICS. [Online]. Available: https://attack.mitre.org/techniques/T0836/

[8] I. Ahmed, S. Obermeier, M. Naedele, and G. G. Richard III, "SCADA systems: Challenges for forensic investigators," Computer, vol. 45, no. 12, pp. 44–51, Dec. 2012.

[9] R. A. Awad, S. Beztchi, J. M. Smith, B. Lyles, and S. Prowell, "Tools, techniques, and methodologies: A survey of digital forensics for SCADA systems," in Proc. 4th Annu. Industrial Control System Security Workshop (ICSS), San Juan, PR, USA, 2018, pp. 1–8, doi: 10.1145/3295453.3295454.

[10] P. Huitsing, R. Chandia, M. Papa, and S. Shenoi, "Attack taxonomies for the Modbus protocols," Int. J. Critical Infrastructure Protection, vol. 1, pp. 37–44, 2008, doi: 10.1016/j.ijcip.2008.08.003.

[11] Wireshark Foundation, "Display Filter Reference: Modbus/TCP" and "Modbus." [Online]. Available: https://www.wireshark.org/docs/dfref/m/mbtcp.html

[12] National Cybersecurity Authority, "Operational Technology Cybersecurity Controls (OTCC-1:2022)," Riyadh, Saudi Arabia, 2022. [Online]. Available: https://nca.gov.sa

[13] Kingdom of Saudi Arabia, "Anti-Cyber Crime Law," Royal Decree No. M/17, 8/3/1428H (2007), Bureau of Experts at the Council of Ministers.

[14] Kingdom of Saudi Arabia, "Personal Data Protection Law," Royal Decree No. M/19 (2021), amended by Royal Decree No. M/148 (2023), Saudi Data and Artificial Intelligence Authority (SDAIA).

[15] S. Davidoff and J. Ham, Network Forensics: Tracking Hackers through Cyberspace. Upper Saddle River, NJ, USA: Prentice Hall, 2012.

Appendix A – Commands Used

Preparation (all VMs, NAT used only for package installation): `sudo apt install -y python3 tcpdump tshark`.

```
# OT-PLC (APPServer)
sudo ip link set enp0s8 up
sudo ip addr add 10.20.30.10/24 dev enp0s8
python3 -u ~/quick_lab.py server | tee plc_console.txt
sudo tcpdump -i enp0s8 -s 0 -w baseline.pcap 'tcp port 1502'
sudo tcpdump -i enp0s8 -s 0 -w incident.pcap 'tcp port 1502'
sha256sum baseline.pcap incident.pcap plc_audit.jsonl plc_console.txt > SHA256SUMS.txt
sha256sum -c SHA256SUMS.txt

# OT-CLIENT (Ubuntu2)
sudo ip link set enp0s8 up
sudo ip addr add 10.20.30.20/24 dev enp0s8
ping 10.20.30.10
for i in 1 2 3; do python3 ~/quick_lab.py read; sleep 1; done      # baseline
python3 ~/quick_lab.py read; python3 ~/quick_lab.py write 90; python3 ~/quick_lab.py read
# incident

# Analysis workstation
sha256sum -c SHA256SUMS.txt
tshark -r baseline.pcap -d tcp.port==1502,mbtcp -Y 'modbus.func_code == 6'
tshark -r incident.pcap -o mbtcp.tcp.port:1502 -Y modbus
tshark -r incident.pcap -o mbtcp.tcp.port:1502 -Y 'frame.number==14' -O mbtcp,modbus
python3 correlate_evidence.py ./Evidence_Artifacts
```

Appendix B – Hash Manifest and Chain of Custody

File	SHA-256
baseline.pcap	557fcdac0c5073177d2b6f279bd45da269c1c51874fa0d86c6501fbb3782973b
incident.pcap	e919a17b09329b853882b9a9d76ca1f4256da8ef87d1a98fbd7cc555dedc25f4
plc_audit.jsonl	fa48dcc0bcd6c439ca11ac4e098b14a519013a16903b2c29ca857840c760fa05
plc_console.txt	c85ff2b1b6bdfe3c45d428301b338e89f4857d90f25a16e9dcf28c691c35d0c6
CourseWork.zip (transfer package)	ab2df80fcf25601cc826657168da6d84de37eff47dafd6023cdc4414cff24623

Table B1. SHA-256 manifest. The first four values were created on OT-PLC; all were re-verified on 11 Nov 2026 (OK).

Step	Date (2026)	Custodian	Action	Location	Integrity check
1	26 Oct	Student 2 Name	Captured E1, E2; simulator wrote E3, E4	OT-PLC ~/sec524_run1	n/a (original)
2	26 Oct	Student 2 Name	Stopped writers; created SHA256SUMS.txt	OT-PLC ~/sec524_run1	sha256sum -c: 4 × OK
3	26 Oct	Student 2 Name	Archived evidence and screenshots as CourseWork.zip	Host laptop	Manifest inside archive

Step	Date (2026)	Custodian	Action	Location	Integrity check
4	11 Nov	Student 1 Name	Received archive from team member; extracted copy	Analysis workstation	sha256sum -c: 4 × OK
5	11 Nov	Student 1 Name	Analysed working copies only; originals kept in archive	Analysis workstation	Hashes re-checked by script: OK

Table B2. Chain-of-custody record.

Appendix C – Raw Decode Output

tshark output with the port preference set (incident.pcap):

```
    4   0.000717  10.20.30.20 → 10.20.30.10  Modbus/TCP 78    Query: Trans: 1; Unit: 1,
Func: 3: Read Holding Registers
    6   0.004993  10.20.30.10 → 10.20.30.20  Modbus/TCP 77 Response: Trans: 1; Unit: 1,
Func: 3: Read Holding Registers
   14  12.631225  10.20.30.20 → 10.20.30.10  Modbus/TCP 78    Query: Trans: 1; Unit: 1,
Func: 6: Write Single Register
   16  12.638099  10.20.30.10 → 10.20.30.20  Modbus/TCP 78 Response: Trans: 1; Unit: 1,
Func: 6: Write Single Register
   24  16.353912  10.20.30.20 → 10.20.30.10  Modbus/TCP 78    Query: Trans: 1; Unit: 1,
Func: 3: Read Holding Registers
   26  16.355162  10.20.30.10 → 10.20.30.20  Modbus/TCP 77 Response: Trans: 1; Unit: 1,
Func: 3: Read Holding Registers
```

Python decoder output, correlation section (correlate_evidence.py):

```
16:27:40.848673 READ          50->50  | request baseline.pcap#4 at +1.83 ms, response #6 at
+0.24 ms, IP match=True
16:27:41.954475 READ          50->50  | request baseline.pcap#14 at +1.70 ms, response #16
at +0.80 ms, IP match=True
16:27:42.991877 READ          50->50  | request baseline.pcap#24 at +5.26 ms, response #26
at +0.21 ms, IP match=True
16:31:17.594328 READ          50->50  | request incident.pcap#4 at +4.69 ms, response #6 at
+0.41 ms, IP match=True
16:31:30.230534 WRITE_APPLIED 50->90  | request incident.pcap#14 at +10.38 ms, response #16
at +3.51 ms, IP match=True
16:31:33.944305 READ          90->90  | request incident.pcap#24 at +1.47 ms, response #26
at +0.22 ms, IP match=True
FC06 frames in baseline.pcap: 0     FC06 frames in incident.pcap: 2
```

The full outputs are provided as supporting files (Analysis/tshark_output.txt and Analysis/correlation_output.txt in the supporting files).

Appendix D – Selected Screenshots

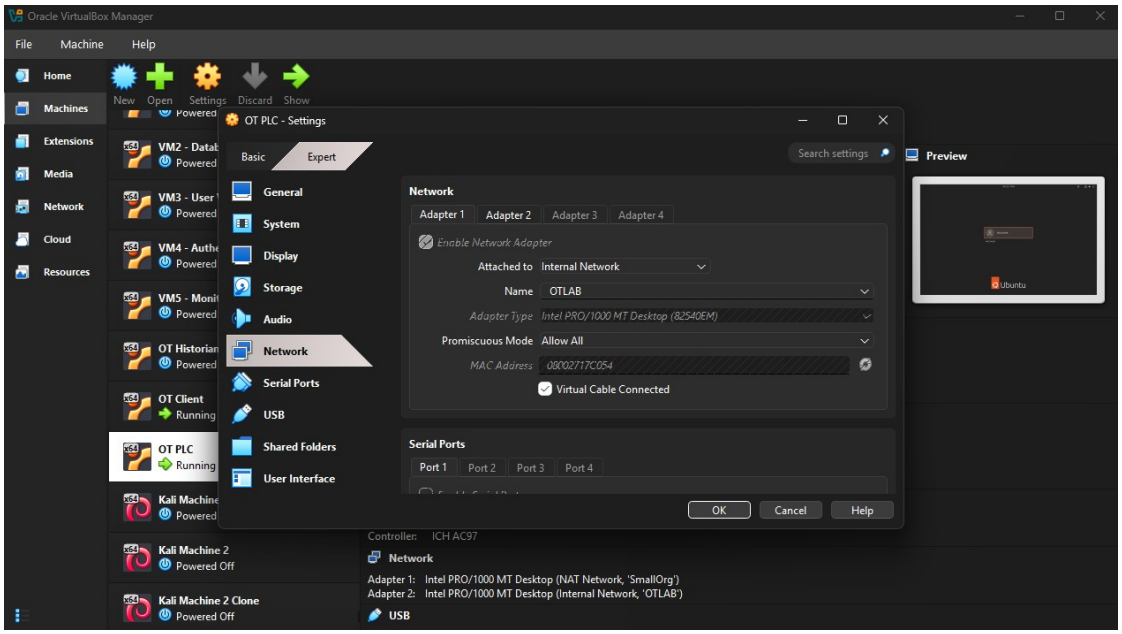


Figure D1. OT-PLC network settings: adapter attached to Internal Network "OTLAB".

```
vboxuser@APPServer:~$ ping 10.20.30.20
PING 10.20.30.20 (10.20.30.20) 56(84) bytes of data.
64 bytes from 10.20.30.20: icmp_seq=1 ttl=64 time=1.22 ms
64 bytes from 10.20.30.20: icmp_seq=2 ttl=64 time=0.737 ms
64 bytes from 10.20.30.20: icmp_seq=3 ttl=64 time=0.917 ms
64 bytes from 10.20.30.20: icmp_seq=4 ttl=64 time=0.540 ms
64 bytes from 10.20.30.20: icmp_seq=5 ttl=64 time=2.07 ms
64 bytes from 10.20.30.20: icmp_seq=6 ttl=64 time=0.695 ms
64 bytes from 10.20.30.20: icmp_seq=7 ttl=64 time=0.531 ms
^C
--- 10.20.30.20 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6176ms
rtt min/avg/max/mdev = 0.531/0.958/2.067/0.503 ms
```

Figure D2. Connectivity test from OT-PLC to OT-CLIENT (0% loss).

```
vboxuser@Ubuntu2:~/sec524_run1$ python3 ~/quick_lab.py read
Register 16 = 50
vboxuser@Ubuntu2:~/sec524_run1$ python3 ~/quick_lab.py write 90
Write acknowledged: 90
vboxuser@Ubuntu2:~/sec524_run1$ python3 ~/quick_lab.py read
Register 16 = 90
```

Figure D3. Client output during the incident: read 50, write 90 acknowledged, read 90.

```
vboxuser@APPServer:~/sec524_run1$ python3 -u ~/quick_lab.py server | tee plc_console.txt
Simulated PLC register 16 = 50; listening at 10.20.30.10:1502
{"time_utc": "2026-10-26T16:27:40.848673+00:00", "client_ip": "10.20.30.20", "event": "READ", "function": 3, "register":
 16, "argument": 1, "before": 50, "after": 50}
{"time_utc": "2026-10-26T16:27:41.954475+00:00", "client_ip": "10.20.30.20", "event": "READ", "function": 3, "register":
 16, "argument": 1, "before": 50, "after": 50}
{"time_utc": "2026-10-26T16:27:42.991877+00:00", "client_ip": "10.20.30.20", "event": "READ", "function": 3, "register":
 16, "argument": 1, "before": 50, "after": 50}
{"time_utc": "2026-10-26T16:31:17.594328+00:00", "client_ip": "10.20.30.20", "event": "READ", "function": 3, "register":
 16, "argument": 1, "before": 50, "after": 50}
{"time_utc": "2026-10-26T16:31:30.230534+00:00", "client_ip": "10.20.30.20", "event": "WRITE_APPLIED", "function": 6, "r
egister": 16, "argument": 90, "before": 50, "after": 90}
{"time_utc": "2026-10-26T16:31:33.944305+00:00", "client_ip": "10.20.30.20", "event": "READ", "function": 3, "register":
 16, "argument": 1, "before": 90, "after": 90}
```

Figure D4. Simulator console showing the six audit records.

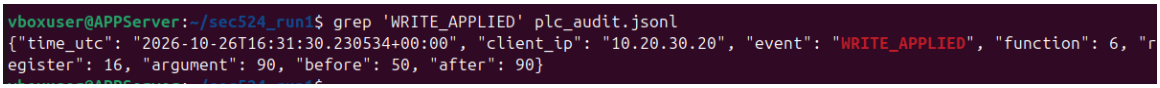


Figure D5. The single WRITE_APPLIED record in plc_audit.jsonl.

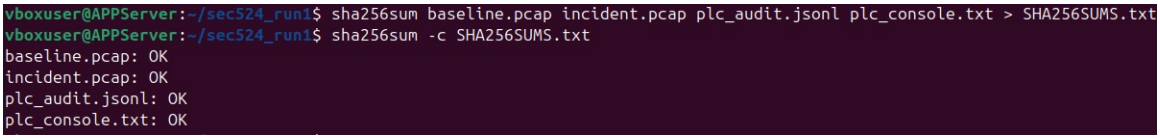


Figure D6. Creation and verification of SHA256SUMS.txt on OT-PLC.

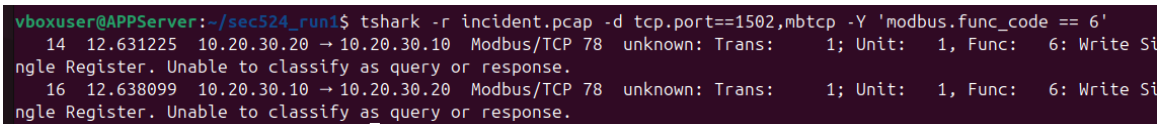


Figure D7. tshark filter for function code 6 on incident.pcap (frames 14 and 16).

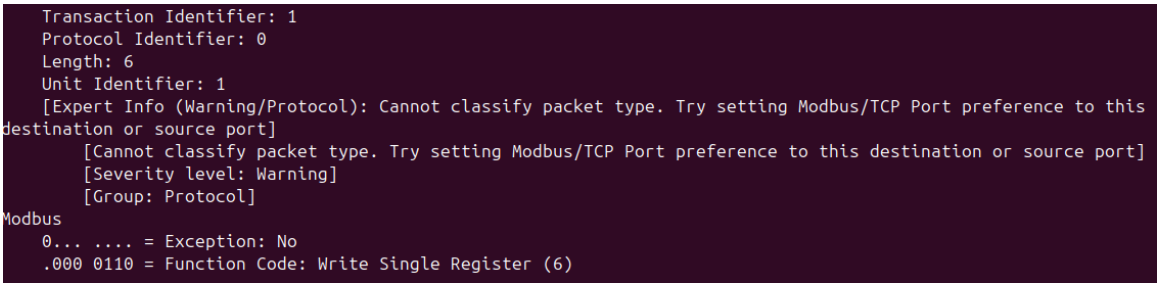


Figure D8. Detailed decode of frame 14 with the dissector warning "Cannot classify packet type" (see Section 6.4).

Appendix E – Supporting Files and AI-Use Statement

Supporting files submitted with this report: the original evidence (E1–E4 and SHA256SUMS.txt), the simulator script quick_lab.py, the lab set-up guide, the analysis script correlate_evidence.py with its output, the tshark output, and all 20 screenshots. To reproduce the analysis, extract the evidence folder, run `sha256sum -c SHA256SUMS.txt`, then run the commands in Appendix A.

AI-use statement: generative AI tools were used, as permitted for this coursework, to help with drafting and editing text, preparing figures and writing the analysis script. The virtual lab, the experiment and the evidence collection were carried out by the team. The team reviewed all results against the original evidence files and takes full responsibility for the content.